# Computational Dialectics Based on Specialization and Generalization – A New Reasoning Method for Conflict Resolution

Hiroyuki Kido and Masahito Kurihara

Graduate School of Information and Science, Hokkaido University

**Abstract.** The purpose of this paper is two-fold. Firstly, to formalize one aspect of dialectical thought, i.e., our way of thinking about conflict resolution. Secondly, to show that this way of thinking allows agents to resolve a conflict by argumentation. To this end, we propose a dialectical reasoning method by means of specialization and generalization defined by a logical implication. This method has three features. First, it does not limit its premises to logical contradictions in accordance with philosophical knowledge that an antithesis is not adequately expressed as logical negation of a thesis. Second, it embraces our actual and familiar thoughts exemplified in this paper. Third, it has the ability to draw conclusions that are not just logical deductions from its premises. Further, by applying it to argumentation, we show that it allows agents to resolve a conflict by drawing an alternative solution not deduced from any consistent subset of the union of all agents' knowledge base. In other words, it allows agents to develop argumentation dialogically in terms of producing an alternative solution that is not obtained at the beginning of argumentation.

## 1 Introduction

Argumentation has its roots in nonmonotonic reasonings of computer science. However, these types of argument, e.g., deduction, induction, abduction, and analogy, and the role of nonmonotonicity in reasonings are different in the case of many other nonmonotonic reasonings and argumentation. The default reasoning which is the most typical approach to realize nonmonotonic reasoning deals only with deductive arguments. Further, it attempts to maintain a consistent source of information by using exceptional rules, i.e., the role of nonmonotonicity in reasoning is conflict avoidance. On the other hand, argumentation deals not only with deductive arguments but also inductive, analogical, and abductive reasonings. Further, it attempts to resolve any conflicts caused by reasoning from subjective, inconsistent, uncertain, imperfect, decentralized, and open sources of information, i.e., the role of nonmonotonicity in reasoning is conflict resolution. Therefore, unlike many other nonmonotonic reasoning approaches, methods to resolve the inevitable contradictions and conflicts are essential requirements of argumentation. In particular, when the purpose of argumentation is to achieve

H. Hattori et al. (Eds.): JSAI 2008, LNAI 5447, pp. 228–241, 2009.
© Springer-Verlag Berlin Heidelberg 2009

social decision making or consensus building, not only competition, but also collaboration, concession, compromise, etc., are essential factors. Hence, in computer science, dialectics has become the focus of attention as guidelines in recent times. This led to the study of computational dialectics, which attempts to analyze dialectics computationally, in computer science [1,2,3].

In the past, Routley and Meyer formalized dialectical logic DL and DM aiming to transform dialectics into a formal logic [5]. Mitroff and Mason constructed a model of dialectical reasoning based on a plausibility measure and Toulmin's model of an argument [2]. Recently, from the view of computer science, Sawamura realized concession and compromise on dialectical logics DL and DM by introducing seven dialectical inference rules [1]. We think, however, these existing studies have three issues with respect to dialectics. First, in [1,2,5], the proposed reasoning methods limit their premises to logical contradictions owing to implicitly, assuming that an antithesis is adequately expressed as a logical negation of a thesis. However, there is a philosophical opinion opposed to the interpretation of the antithesis [3]. Second, in [2,5], the authors showed no concrete example of thought they tried to model. Third, in [1], the dialectical inference rules 1- 6 discussed later are weak in the sense that they do not have the ability to draw new propositions not appearing in the original premise.

In consideration of these issues, we formalize one aspect of dialectical thought as a reasoning method by means of specialization and generalization defined by a logical implication. It has three features. First, it does not limit its premises to logical contradictions in accordance with philosophical knowledge that dialectical negation is an antithesis (counterplan), i.e., an alternative solution not adequately expressed as a logical negation of a thesis. Second, it embraces our actual and familiar thoughts exemplified in this paper. Third, it has the ability to draw conclusions that are not just logical deductions from its premise. Further, by applying it to argumentation, we show that it allows agents to resolve a conflict through drawing an alternative solution not deduced from any consistent subset of the union of all agents' knowledge base. In other words, it allows agents to develop argumentation dialogically in terms of producing an alternative solution that is not obtained at the beginning of argumentation.

Section 2 briefly discusses dialectics and the existing studies of computational dialectics. In section 3, we give the algorithm for dialectical reasoning and its strategies. In section 4, we define collaboration, concession, and compromise. In section 5, we introduce an argumentation model basically defined in [9]. In section 6, we provide an application of conflict resolution by applying dialectical reasoning in argumentation. In section 7, we contrast the proposed method with existing computational dialectics. In section 8, the conclusion is given.

## 2 Dialectics

### 2.1 What Is Dialectics?

Etymologically speaking, dialectics means 'dialogue', 'skills of dialogue', etc. Psychologist Richard interpreted dialectics as having the meaning of focusing

on contradictions and how to resolve them, to transcend them, or to find the truth in both. Furthermore, in his studies, he contrasted a logical approach with the dialectical approach for conflicting propositions: The logical approach would seem to require rejecting one of the propositions in favor of the other in order to avoid possible contradiction. The dialectical approach would favor finding some truth in both, in a search for the Middle Way [4]. From this standpoint, we think our thought called collaboration, concession, and compromise are one of the dialectical approaches for this conflict. At the risk of doing damage to the spirit of dialecticism, we generally call these dialectical thoughts. In addition, from this standpoint we provide a typical examples of dialectical thinking from every day life.

*Example 1 (purchase of a camera)*

- $A_1$: I want to buy a compact and light camera 'a.'
- $A_2$: We can not buy it, because it is out of stock.
- $A_2$: I want to buy a high-resolution camera 'b' with a long battery life.
- $A_1$: No, I do not want to buy it, because it is beyond my budget.
- $A_1$: Then let us buy a user-friendly camera 'c' with a long battery life.

This is because $A_1$ knows it is user-friendly if it is compact and light.

*Example 2 (menu decision)*

- $A_1$: I want to eat a curry rice 'a.'
- $A_2$: No, I do not want to eat it.
- $A_2$: I want to eat Chinese noodles 'b.'
- $A_1$: No, I do not want to eat it.
- $A_1$: Then let us eat curry noodles 'c.'

This is because $A_1$ knows curry rice is a curry, Chinese noodles are noodles, and curry noodles are a curry and noodles.

### 2.2 Existing Studies on Computational Dialectics

Routley and Meyer formalized dialectical logic DL and DM with the aim to transform dialectics into formal logic [5]. Mitroff and Mason constructed a model of dialectical reasoning based on a plausibility measure and Toulmin's developed a model of an argument [2]. It computes all maximal consistent subsets of an inconsistent set of formulae constituting the argument, and applies it based on the dialectical plausibility measure assigned to each. Recently, from the view of computer science, Sawamura identified dialectics as a new object of study that is different from other studies of reasoning such as deduction, induction, abduction, and analogy. He realized concession and compromise on DL and DM[1] as formal logic by introducing the dialectical inference rules as follows. They rationally enable to draw a satisfiable conclusion from inconsistent premises.

[1] Refer to [5] for detailed definitions of DL and DM.

1. $A, \neg A \Rightarrow A$
2. $A, \neg A \Rightarrow \neg A$
3. $A \wedge B, \neg B \Rightarrow A$
4. $A \wedge B, \neg B \Rightarrow B$
5. $A \wedge B, \neg B \Rightarrow A \wedge \neg B$
6. $A \wedge \neg B, \neg A \wedge B \Rightarrow A \wedge B$
7. $A(a), \neg A(a) \Rightarrow A(a) \wedge \neg A(b)$

However, it is thought that these existing studies have three issues with respect to dialectics. First, in [1,2,5], the proposed reasoning methods limit their premises to logical contradiction owing to implicitly assuming that an antithesis is logical negation as a thesis. However, there is a philosophical opinion opposed to the interpretation of an antithesis as a logical negation as a thesis: This is severely hampered by the interpretation of the antithesis (counterplan) as logical negation of a thesis, because this prevents a full positive statement of the antithesis and provides no place for the rationale of the antithesis [3]. Second, in [2,5], the authors showed no concrete example of thought they tried to model. Third, in [1], the dialectical inference rules 1- 6 are weak in the sense that they do not have the ability to draw new propositions not appearing in the original premise.

In consideration of these issues, we formalize one aspect of dialectical thought as a reasoning method by means of specialization and generalization defined by a logical implication. It has three features. First, it does not limit its premises to logical contradiction in accordance with philosophical knowledge that a dialectical negation is an antithesis (counterplan), i.e., an alternative solution is not adequately expressed as logical negation as a thesis. Second, it embraces our actual and familiar thoughts exemplified in this paper. Third, it has the ability to draw conclusions that are not just logical deductions from its premise. Further, by applying it to argumentation, we show that it allows agents to resolve conflicts through drawing alternative solutions not deduced from any consistent subset of the union of all agents' knowledge base. In other words, it allows agents to develop argumentation dialogically in terms of producing alternative solutions that are not obtained at the beginning of argumentation.

## 3 Reasoning Method

### 3.1 Algorithm for Dialectics

In this section, we formalize one aspect of dialectical thought by capturing our thought on conflict resolution. First, we show typical examples representing our thought process in conflict resolution.

*Example 3 (process of drink decision)*

- $A_1$ : I want to drink a cup of black tea. ($p_1 = drink(a) \wedge blackTea(a)$)
- $A_2$ : I want to drink a cup of green tea. ($p_2 = drink(b) \wedge greenTea(b)$)
- $A_1$ : I intend to stand my ground. ($p_3 = p_1$)

- $A_2$ : Okay, either will be fine. ($p_4 = p_1 \vee p_2$)
- $A_1$ : Thanks! ($p_5 = p_1$)

*Example 4 (process of menu decision)*

- $A_1$ : If it is curry rice then I will eat it. ($p_1 = \forall x.eat(x) \leftarrow curriedRice(x)$)
- $A_2$ : If it is Chinese noodles then I will eat it. ($p_2 = \forall x.eat(x) \leftarrow chineseNoodle(x)$)
- $A_1$ : It might be a good idea to eat it if it is a noodles. ($p_3 = p_1 \vee \forall x.eat(x) \leftarrow noodle(x)$)
- $A_2$ : It also might be a good idea to eat it if it is curry. ($p_4 = p_2 \vee \forall x.eat(x) \leftarrow curry(x)$)
- $A_1$ : Then let us eat it if it is noodles and curry. ($p_5 = \forall x.eat(x) \leftarrow noodle(x) \wedge curry(x)$)

In example 3 and 4, it should be noted that $p_3$, $p_4$ and $p_5$ are fully implemented only by specialization and generalization defined by a 'logical implication' relation. In addition, the process of calculation fits into our thought process on conflict. Actually, in example 3, $p_3$ is a specialization of $p_1$, i.e., $p_1 \models p_3$, $p_4$ is a specialization of $p_2$, i.e., $p_2 \models p_4$, and $p_5$ is generalization of $p_3$ and $p_4$, i.e., $p_5 \models p_3$ and $p_5 \models p_4$. In example 4, $p_1 \models p_3$, $p_2 \models p_4$, $\{p_5\} \cup \Delta \models p_3$, and $\{p_5\} \cup \Delta \models p_4$ hold under background knowledge $\Delta = \{curry(x) \leftarrow curryRice(x), noodles(x) \leftarrow chineseNoodles(x)\}$.

Example 3 and 4 do not explain why $A_1$ and $A_2$ yield in such a manner. In this paper, we initially focus on only the logical aspect of the process of conflict resolution that is normally influenced by various factors, e.g., emotion of the main parties. We think that the reason and the degree of yielding depend strongly on such factors. Therefore, they are not reflected in the following algorithm. In the next subsection, however, we define reasoning strategies based on the two reasonable assumptions related to these factors to restrict the reasoning. First, we give a reasoning algorithm for dialectics from a logical point of view. The inputs of the algorithm are contingent first-order formulae $A$ and $B$, and the set of first-order formulae $\Gamma$ such that both $\{A\} \cup \Gamma$ and $\{B\} \cup \Gamma$ are consistent. The output is a set by first-order formulae $\mathcal{C}$. Algorithm 1 consists of specialization at line 4-5 and generalization at line 9.

Figure 1 represents the logical relation between formulae appearing in the algorithm. The nodes represent the formulae and edges represent logical relation between them. The dialectics follows the arrows.

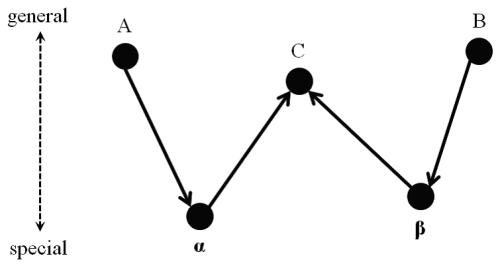



**Fig. 1.** dialectical reasoning

**Algorithm 1.** Dialectical Reasoning

1: $\mathcal{C} := \emptyset$
2: $\alpha := null$
3: $\beta := null$
4: computes $\mathcal{A} \subseteq \{\alpha \mid \text{not } \Gamma \models \alpha, \{A\} \cup \Gamma \models \alpha\}$
5: computes $\mathcal{B} \subseteq \{\beta \mid \text{not } \Gamma \models \beta, \{B\} \cup \Gamma \models \beta\}$
6: **for all** $(\alpha, \beta)$ in $\mathcal{A} \times \mathcal{B}$ **do**
7: $\quad \mathcal{C}_i := \emptyset$
8: $\quad C := null$
9: $\quad$ computes $\mathcal{C}_i \subseteq \{C \mid \{C\} \cup \Gamma \models \alpha, \{C\} \cup \Gamma \models \beta\}$
10: $\quad \mathcal{C} := \mathcal{C} \cup \mathcal{C}_i$
11: **end for**
12: return $\mathcal{C}$

### 3.2 Reasoning Strategies

We introduce some strategies of the reasoning defined in the previous subsection. When we face a conflict, we usually try to search for a solution that takes into account our intentions as much as possible and does not take into account unintentional considerations. Further, if agents show willingness to compromise, they meet each other halfway toward each other. We capture these points computationally from the view of the degree and the direction of specialization and generalization.

**Specialization.** At line 4 and 5 in algorithm 1, agents compute $\alpha$ and $\beta$ such that $A \models \alpha, \{\alpha\} \cup \Gamma \models \gamma, B \models \beta$, and $\{\beta\} \cup \Gamma \models \gamma$, where $\gamma$ is the first-order formula representing the greatest lower bound of set $\{A, B\}$. It is $A \vee B$ in the case of first-order logic. These conditions are related to the direction of the specialization. On the other hand, for the degree of specialization, agents compute the specialization of $A$ and $B$, i.e., $\alpha$ and $\beta$, by using downward refinement operator defined in the next subsection that enables the calculation of specialization step by step.

**Generalization.** At line 9 in algorithm 1, agents compute least upper bound of the set $\{\alpha, \beta\}$. It is $A \wedge B$ in the case of first-order logic. However, it is not necessarily in the case of clausal logic. This condition is related to both the direction and the degree of generalization.

*Example 5.* In example 3 and 4, every $p_3$, $p_4$, and $p_5$ complies with the strategies in specialization and generalization.

### 3.3 Algorithms for Specialization and Generalization

Subsequently, in this section, by limiting the target of computing to definite program clauses, we are going to show some concrete algorithms to compute specialization and generalization based on the quasi-order on clauses with background knowledge, i.e., Buntine's generalized subsumption [8], respectively. We

are going to build the above strategies into the algorithms based on the results of inductive logic programming.

Let $A$ and $\alpha$ be definite program clauses and $\Delta$ be a definite program. Then it is noted that $\alpha$ is a specialization of $A$ relative to $\Delta$ under generalized subsumption (denoted by $A \geq_\Delta \alpha$) iff there exists an SLD-deduction of $\alpha$, with $A$ as top clause and members of $\Delta$ as input clauses [8]. Based on this, we provide concrete algorithm computing specialization under generalized subsumption. The inputs of the algorithm are the definite program clause $A$ and the definite program $\Delta$. The output is the definite program $\mathcal{A} \subseteq \{\alpha \mid A \geq_\Delta \alpha\}$.

**Algorithm 2.** Downward refinement operator

1: $\mathcal{A} := \rho_L(A)$
2: **for all** $(A, \delta)$ in $\{A\} \times \Delta$ **do**
3: computes binary resolvent $\alpha$ of $A$ and $\delta$
4: **if** $\alpha$ is computable **then**
5: $\mathcal{A} := \mathcal{A} \cup \{\alpha\}$
6: **end if**
7: **end for**
8: return $\mathcal{A}$

Algorithm 2 is a downward refinement operator for $< \mathcal{C}, \geq_\Delta >$, where $\mathcal{C}$ is a clausal language, such that $\rho(A, \Delta) \subseteq \{\alpha \mid A \geq_\Delta \alpha\}$ for every $A \in \mathcal{C}$. The set of one-step refinements, n-step refinements, and refinements of some $A$ are respective:

- $\rho^1(A, \Delta) = \rho(A, \Delta)$
- $\rho^n(A, \Delta) = \{\alpha \mid$ there is an $\beta \in \rho^{n-1}(A, \Delta)$ such that $\alpha \in \rho(\beta, \Delta)\}$, $n \geq 2$
- $\rho^*(A, \Delta) = \rho^1(A, \Delta) \cup \rho^2(A, \Delta) \cup \cdots$

$\rho_L$ at line 1 in the above algorithm is also a downward refinement operator for $< \mathcal{C}, \succeq >$, where $\succeq$ is a subsumption relation, that computes the specialization of the clause by applying several different kinds of substitutions or adding new literal to the clause [8]. For every combination $(A, \delta) \in \{A\} \times \Delta$, $\rho$ attempts to compute the binary resolvent $\alpha$ of $A$ and $\delta$, and adds it to the set $\mathcal{A}$ if it is computable. Next, we give the concrete algorithm combining the strategies for specialization. The inputs to the algorithm are definite program clauses $A$ and $B$, the definite program $\Delta$, and a number of refinements steps $n$. The output is a definite program $\mathcal{A} \subseteq \{\alpha \mid A \geq_\Delta \alpha\}$. Algorithm 3 uses algorithm 2 along with a number of refinements steps $n$ to restrict the degree of specialization. The algorithm accumulates the program clauses satisfying the strategies in the specialization.

Next we give the concrete algorithm to compute generalization of the set of the specialized program clauses. We compute the least generalization under generalized subsumption (denoted by LGGS) of the set of definite program clauses in accordance with the strategy of generalization. It should be noted that the generalized subsumption can be translated to ordinary subsumption, and the

**Algorithm 3.** Specialization

1: $\mathcal{A} := \emptyset$
2: **for** $i := 1$ to $n$ **do**
3: **for all** $\alpha$ in $\rho^i(A, \Delta)$ **do**
4: **if** $\{\alpha\} \cup \Delta \models A \vee B$ **then**
5: $\mathcal{A} := \mathcal{A} \cup \{\alpha\}$
6: **end if**
7: **end for**
8: **end for**
9: return $\mathcal{A}$

LGGS can be computed by constructing a least generalization under the subsumption (denoted by LGS) [8]. In the following algorithm, it is particularity helpful to denote that the positive literal of the clause $D$ by $D^+$, the set of negative literals of the clause $D$ by $D^-$, and the set of negations of the formulae in the set $M$ by $\overline{M}$. The inputs of the algorithm are definite program clauses $D_1$ and $D_2$ such that they have the same predicate symbol as their head, and the definite program $\mathcal{B}$. The output is the LGGS of $\{D_1, D_2\}$.

**Algorithm 4.** Generalization

1: computes the skolem substitutions $\sigma_i (i = 1, 2)$ for $D_i$ with respect to $\mathcal{B} \cup \{D_1, D_2\}$
2: computes the least Herbrand models $M_i (i = 1, 2)$ of $\mathcal{B} \cup D_i^- \sigma_i$
3: computes the LGS $C$ of $\{\{D_1^+ \sigma_1\} \cup \overline{M_1}, \{D_2^+ \sigma_2\} \cup \overline{M_2}\}$
4: return $C$

For each $D_i (i = 1, 2)$, algorithm 4 computes the skolem substitution $\sigma_i$ and the Herbrand model $M_i$ to treat generalized subsumption as subsumption. The algorithm indirectly computes the LGGS of $\{D_1, D_2\}$ based on the LGS of $\{\{D_1^+ \sigma_1\} \cup \overline{M_1}, \{D_2^+ \sigma_2\} \cup \overline{M_2}\}$.

Next example shows the calculation process of algorithm 1 combined with the above algorithms 3 and 4.

*Example 6 (dialectical reasoning).* Consider the following two clauses $C_1$ and $C_2$, and background knowledge $\mathcal{B}$.

$$C_1 = buy(x) \vee \neg camera(x) \vee \neg compact(x) \vee \neg light(x)$$
$$C_2 = buy(x) \vee \neg camera(x) \vee \neg resolution(x, high) \vee \neg battery(x, long)$$
$$\mathcal{B} = \{userFriendly(x) \leftarrow compact(x) \wedge light(x)\}$$

Following $D_1$ and $D_2$ are the outputs of algorithm 3 whose inputs are $C_1$, $C_2$, $\mathcal{B}$, and 3, for $D_1$, and $C_2$, $C_1$, $\mathcal{B}$, and 2, for $D_2$, respectively.

$$D_1 = buy(x) \vee \neg camera(x) \vee \neg compact(x) \vee \neg light(x) \vee \neg battery(x, long)$$
$$D_2 = buy(x) \vee \neg camera(x) \vee \neg resolution(x, high) \vee \neg battery(x, long) \vee \neg userFriendly(x)$$

$D_1$ is derived from $C_1$ by adding the literal $\neg battery(y, z)$ and applying the substitutions $\{y/x\}$ and $\{z/long\}$ to $C_1$ in accordance with $\rho_L$.[2] $D_2$ is derived from $C_2$ by adding the literal $\neg userFriendly(y)$ and applying the substitution $\{y/x\}$ to $C_2$. Let us assume $\sigma_1 = \{x/a\}$ and $\sigma_2 = \{x/b\}$ to be Skolem substitutions for $D_1$ and $D_2$, respectively. Then

$$M_1 = \{camera(a), compact(a), light(a), battery(a, long), userFriendly(a)\}$$
$$M_2 = \{camera(b), resolution(b, high), battery(b, long), userFriendly(b)\}$$

Following $E$ is an LGS of $\{(\{D_1^+\sigma_1\} \cup \overline{M_1}), (\{D_2^+\sigma_2\} \cup \overline{M_2})\}$, and hence also an LGGS of $\{D_1, D_2\}$. This means $E$ is the output of algorithm 4 whose inputs are $D_1$, $D_2$, and $\mathcal{B}$.

$$E = buy(x) \vee \neg camera(x) \vee \neg battery(x, long) \vee \neg userFriendly(x)$$

It is noteworthy that $\{C_1, C_2\} \cup \mathcal{B} \models E$ does not hold true, i.e., algorithm 1 can produce formulae not logically deduced from premise.

## 4 Semantics

### 4.1 Collaboration, Concession, and Compromise

The results of the negotiation studies showed relationships among collaboration, concession, compromise, and avoidance or postponement as shown in Figure 2. Each axis represents the degree of consideration of the result. Value 100 means that the result was taken into full account, while a value 0 means that it was not taken into account at all. Based on Figure 2, we define collaboration, concession, and compromise as follows.

**Definition 1.** *(collaboration, concession, and compromise). Let $C$ be a output of algorithm 1 whose inputs are contingent formulae $A$ and $B$, and a set of formulae $\Delta$ such that both $\{A\} \cup \Delta$ and $\{B\} \cup \Delta$ are consistent.*

- *$C$ is collaboration between $A$ and $B$ iff $\{C\} \cup \Delta \models A$ and $\{C\} \cup \Delta \models B$.*
- *$C$ is concession between $A$ and $B$ (or $B$ and $A$) iff $\{C\} \cup \Delta \models A$ and not $\{C\} \cup \Delta \models B$.*
- *$C$ is compromise between $A$ and $B$ iff neither $\{C\} \cup \Delta \models A$ nor $\{C\} \cup \Delta \models B$.*

*Example 7.* In example 3, $p_5$ is a concession between $p_1$ and $p_2$, and, in example 4, $p_5$ is a compromise between $p_1$ and $p_2$. Further, in example 6, $E$ is a compromise between $C_1$ and $C_2$.

[2] We can not show the detailed definitions of $\rho_L$ due to limitations of space. Refer to [8] for details.

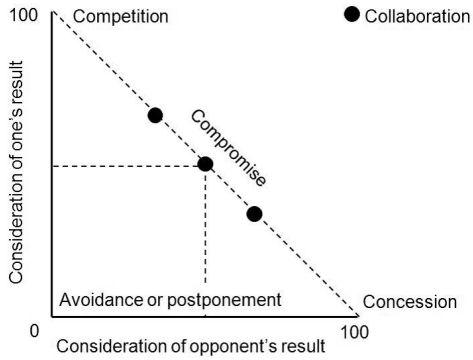



**Fig. 2.** Competition, collaboration, concession, compromise, and avoidance or postponement [6,7]

## 5 Argumentation Model

We basically use Prakken and Sartor's argumentation framework [9]. A defeasible rule, or simply rule, is an expression of the form '$r : L_1 \wedge \cdots \wedge L_j \wedge \sim L_{j+1} \wedge \cdots \wedge \sim L_{n-1} \Rightarrow L_n$', where every $L_i (1 \leq i \leq n)$ is an atomic first-order formula or its classical negation [9]. For each $\sim L_i (j+1 \leq i \leq n-1)$ in $r$, it reads as 'there is no evidence that $L$' and $\neg L_i$ is called an assumption of $r$. When no assumption exists in the rule we call it a strict rule and use '$\rightarrow$' for the expression. Further, the conjunction at the left of the arrow is called an antecedent of the rule and the literal at the right of the arrow is called a consequent of the rule. When no antecedent exists in the rule we call it a fact and omit the arrow.

**Definition 2.** *An argument is a finite sequence $A = [r_1, \cdots, r_n]$ of ground instances of rules such that:*

1. *For every $i (1 \leq i \leq n)$, for every strong literal $L_j$ in the antecedent of $r_i$ there is a $k < i$ such that $L_j$ is the consequent of $r_k$.*
2. *For every $i (1 \leq i \leq n-1)$, there is a $i < k$ such that the consequent of $r_i$ is the strong literal in the antecedent of $r_k$.*
3. *No two distinct rules in the sequence have the same consequent.*

The key difference between above definition and Prakken and Sartor's is the addition of condition 2. It ensures that argument is a sequence of the minimum number of rules for deducing consequent of $r_n$. For any argument $A = [r_1, \cdots, r_n]$, $Conc(A)$ and $Ass(A)$ denote the set of the consequents of rules in $A$ and a set of the assumptions of rules in $A$, and thus we call them a conclusion of $A$ and an assumption of $A$, respectively. Further, we call $r_n$ a warrant of $A$.

Rebutting is a symmetrical relation and undercutting is an unsymmetrical relation on the set of arguments.

**Definition 3.** *Let $A_1$ and $A_2$ be arguments.*

- *$A_1$ rebuts $A_2$ iff $L \in Conc(A_1)$ and $\neg L \in Conc(A_2)$*
- *$A_1$ undercuts $A_2$ iff $L \in Conc(A_1)$ and $\neg L \in Ass(A_2)$*

If an argument rebuts or undercuts itself we call it incoherent. Based on the rebutting and undercutting, defeat is defined as a relation on the set of arguments.

**Definition 4.** [9] *Let $A_1$ and $A_2$ be two arguments. Then $A_1$ defeats $A_2$ iff $A_1$ is empty and $A_2$ is incoherent, or else if*

- *$A_1$ undercuts $A_2$; or*
- *$A_1$ rebuts $A_2$ and $A_2$ does not undercut $A_1$.*

*We say that $A_1$ strictly defeats $A_2$ iff $A_1$ defeats $A_2$ and $A_2$ does not defeat $A_1$.*

*Example 8.* Let $Arg_1 = [p(a), p(a) \wedge \sim q(a) \Rightarrow r(a)]$ and $Arg_2 = [q(a), q(a) \rightarrow \neg r(a)]$. Then $Arg_1$ and $Arg_2$ are arguments and $Arg_2$ strictly defeats $Arg_1$.

**Definition 5.** *Let $A$ be an argument.*

- *$A$ is justified if no argument defeats $A$ or if every argument which defeats $A$ is also strictly defeated.*
- *$A$ is overruled if $A$ is defeated by the justified argument.*
- *$A$ is defensible if $A$ is neither justified nor overruled.*

## 6 Application in Argumentation-Based Negotiation

We can find solutions related to the issues in argumentation by increasing our understanding of them through discussing opposing viewpoints in the course of argumentation. Such argumentation involves change, revision, or a leap of thinking about our opinions associated with deepening of understanding. The argumentation model in the previous section focuses on not how our opinions can be changed, but what opinions are justified through interaction of the opposing viewpoints. Our dialectical reasoning focuses on the latter and, therefore, can provide guidelines for developing argumentation. In this section, we apply it to conflict resolution as shown in Example 1. We assume following settings for argumentation.

- Every arguing agent $i$ has theory $S_i$, where it contains only strict rules. The theory is unchanged during argumentation.
- There is an agenda 'Issue' of argumentation at the start of the argumentation. It is represented as the conjunction of literals and is unchanged during argumentation.
- Argumentation terminates when there is no main argument that is not presented before or some main argument is justified.

Argumentation consists of two phases. In phase 1, agents make their main arguments whose conclusions have the ground instance of the issue. In phase 2, they make arguments defeating opponent's arguments. In each phase, they make arguments from their own theory. In phase 1, if agents can make no argument not yet presented, then they create new strict rules by applying the proposed reasoning method and make new main arguments based on it. In the following

example, agents apply the proposed reasoning method to the warrants of the mutually defeating arguments. In phase 2, tenability of the main argument is evaluated. This is shown as follows where $Arg_i^j$ denotes argument $i$ made by agent $j$.

$$
\begin{aligned}
Issue &= buy(x) \wedge camera(x) \\
S_1 &= \{camera(a), camera(c), compact(a), light(a), battery(c, long), \\
&\quad userFriendly(c), overTheBudget(b), \\
&\quad compact(x) \wedge light(x) \wedge camera(x) \rightarrow buy(x), \\
&\quad overTheBudget(x) \rightarrow \neg buy(x), \\
&\quad compact(x) \wedge light(x) \rightarrow userFriendly(x)\} \\
S_2 &= \{camera(b), outOfStock(a), battery(b, long), resolution(b, high), \\
&\quad resolution(x, high) \wedge battery(x, long) \wedge camera(x) \rightarrow buy(x), \\
&\quad outOfStock(x) \rightarrow \neg buy(x)\}
\end{aligned}
$$

First, agent 1 tries to make a main argument and agent 2 tries to defeat it.

**Phase 1:** $Arg_1^1 = [compact(a), light(a), camera(a), compact(a) \wedge light(a) \wedge camera(a) \rightarrow buy(a)]$ (I want to buy 'a.' Since it is a compact and light camera.)

**Phase 2:** $Arg_2^2 = [outOfStock(a), outOfStock(a) \rightarrow \neg buy(a)]$ (It is out of stock.)

Agent 1 can not make the argument defeating $Arg_2^2$. Next, agent 2 tries to make a main argument and agent 1 tries to defeat it.

**Phase 1:** $Arg_3^2 = [resolution(b, high), battery(b, long), camera(b), resolution(b, high) \wedge battery(b, long) \wedge camera(b) \rightarrow buy(b)]$ (Would you like 'b?' Because it is a high-resolution camera with a long battery life.)

**Phase 2:** $Arg_4^1 = [overTheBudget(b), overTheBudget(b) \rightarrow \neg buy(b)]$ (It exceeds the budget.)

Main arguments $Arg_1^1$ and $Arg_3^2$ are not justified. Furthermore, both agents can not make another main argument. Then agent 1 can make main argument $Arg_5^1$ whose warrant is a compromise between the warrants of $Arg_1^1$ and $Arg_3^2$.

**Phase 1:** $Arg_5^1 = [userFriendly(c), battery(c, long), camera(c), userFriendly(c) \wedge battery(c, long) \wedge camera(c) \rightarrow buy(c)]$ (I will then buy 'c.' Since this is a user-friendly camera with a long battery life.)

Agent 2 can not make the argument defeating $Arg_5^1$. Therefore, it is justified. Note that any argument from $S_1 \cup S_2$ can not have $buy(c)$ in its conclusion. In other words agents can develop argumentation dialogically by producing an alternative solution that is not drawn at the beginning of the argumentation.

## 7 Discussion

In this section, we contrast the proposed reasoning method with existing computational dialectics. The dialectical reasonings proposed in [1,2] adopted logical contradiction as the premises of the reasoning. In contrast, our method does not limit its premises to logical contradiction. Therefore, not only logical contradiction, but also conflicts or differences caused by the existence of alternative solutions can be the object of computing. Further, algorithm 1 has the ability to draw propositions derived by applying dialectical inference rules 1-7 in [1]. In addition, our method can draw conclusions which are not derived from them. Example 6 is one such example.

In section 4, we define concession, compromise, and cooperation. Our definitions are contrast to the following definition.

**Definition 6.** [1] *(Compromise/Aufheben). Given two conflicting propositions, $A$ and $B$, a proposition $C$ is said to be a higher-order agreement (Aufheben) lifted up from $A$ and $B$ or simply a compromise if (i) neither $\vdash A \rightarrow C$ nor $\vdash B \rightarrow C$, and (ii) $C$ shares some atomic propositions with $A$ or $B$.*

**Definition 7.** [1] *(Concession/weaker Aufheben). Given two conflicting propositions, $A$ and $B$, a proposition $C$ is said to be a higher-order agreement (weaker Aufheben) lifted up from $A$ and $B$ or simply a concession if (i) it is not the case $\vdash A \rightarrow C$ and it is the case that $\vdash B \rightarrow C$, or it is the case that $\vdash A \rightarrow C$ and it is not the case that $\vdash B \rightarrow C$, and (ii) $C$ shares some atomic propositions with $A$ or $B$.*

The condition (i) in definition 6 means that compromise $C$ is not logically deduced from either $A$ and $B$, and semantically, it means that compromise is not logically of less value than its premise. On the other hand, definition 1 means that compromise $C$ does not imply both $A$ and $B$ with respect to background knowledge $\Delta$, i.e., means that compromise is not logically of greater value than its premise. This difference is due to the alignment of Figure 2 and, similarly, it can be seen in the definition of concession.

## 8 Conclusion

We formalized one aspect of dialectical thought as a reasoning method based on specialization and generalization defined by a logical implication. It has three features: First, it does not limit its premise to logical contradiction. It targets any contingent formulae based on the understanding that an antithesis encompasses conflicts and differences not adequately expressed as a logical negation. Second, it embraces our actual and familiar thoughts exemplified in this paper. We showed concrete thought seen in electronic commerce. Third, it has the ability to draw conclusions that are not just logical deduction from its premise. Further, by applying this reasoning method to argumentation, we show that it allows agents to resolve a conflict through drawing an alternative solution not deduced from

any consistent subset of the union of all agents' knowledge base. In other words, it allows agents to develop argumentation dialogically in terms of producing an alternative solution that are not obtained at the beginning of argumentation.

## References

1. Sawamura, H., Yamashita, M., Umeda, Y.: Applying dialectic agents to argumentation in e-commerce. Electronic Commerce Research 3(3-4), 297–313 (2003)
2. Mitroff, I.I., Mason, R.O.: On the structure of dialectical reasoning in the social and policy sciences. Theory and Decision 14(4), 331–350 (1982)
3. Sabre, R.M.: An alternative logical framework for dialectical reasoning in the social and policy sciences. Theory and Decision 30(3), 187–211 (1991)
4. Nisbett, R.E.: The Geography of thought: How Asians and Westerners Think Differently..and Why. Free Press (2003)
5. Routley, R., Meyer, R.K.: Dialectical logic, classical logic, and the consistency of the world. Studies in East European Thought 16(1-2), 1–25 (1976)
6. Dean, G.P., Jeffrey, Z.R.: Social Conflict: Escalation, Stalemate and Settlement. Random House (1986)
7. Nozawa, S.: Negotiation study for conflict resolution. PHP Laboratory (2004) (in Japanese)
8. Nienhuys-Cheng, S.H., de Wolf, R.: Foundation of Inductive Logic Programming. Springer, Heidelberg (1997)
9. Prakken, H., Sartor, G.: Argument-based extended logic programming with defeasible priorities. Applied Non-Classical Logics 7(1), 25–75 (1997)